“Linux Process Creation, Execution and Termination System”

A Project Report Submitted in the partial fulfillment of the requirements for the award of the degree of

BACHELOR OF TECHNOLOGY

In

DEPARTMENT OF COMPUTER SCIENCE ENGINEERING

By

ILI HARSHIKA	**2520030607**
GURRAPU NIKSHITHA	**2520030512**
ANGELA JOSE	**2520030365**

Under the Esteemed Guidance of

Dr. D. Bheekya

Faculty, Operating Systems and Systems Programming

Department of Computer Science and Engineering



Koneru Lakshmaiah Education Foundation

(Deemed to be University estd. u/s. 3 of the UGC Act, 1956)
Off-Campus: Bachupally-Gandimaisamma Road, Bowrampet, Hyderabad, Telangana - 500 043.
Phone No: 7815926816, www.klh.edu.in

K L (Deemed to be) University
DEPARTMENT OF COMPUTER SCIENCE ENGINEERING

&

DEPARTMENT OF COMPUTER SCIENCE AND INFORMATION TECHNOLOGY



Declaration

The Project Report entitled **"Linux Process Creation, Execution and Termination System"** is a record of Bonafide work of **ILI HARSHIKA – 2520030607, GURRAPU NIKSHITHA – 2520030512, ANGELA JOSE – 2520030365** submitted in partial fulfillment for the award of B. Tech in Computer Science and Engineering (or) Computer Science and Information Technology to the K L University. The results embodied in this report have not been copied from any other departments/University/Institute.

ILI HARSHIKA	**2520030607**
GURRAPU NIKSHITHA	**2520030512**
ANGELA JOSE	**2520030365**

K L (Deemed to be) University

DEPARTMENT OF COMPUTER SCIENCE ENGINEERING



CERTIFICATE

This is to certify that the mini project based report entitled **"Linux Process Creation, Execution and Termination System"** is a bonafide work done and submitted by **Ili Harshika – 2520030607, Gurrapu Nikshitha – 2520030512, Angela Jose – 2520030365** in partial fulfillment of the requirements for the award of the degree of **BACHELOR OF TECHNOLOGY** in Department of Computer Science Engineering, K L (Deemed to be University), during the academic year **2026-2027.**

Signature of the Guide

Signature of the Course Coordinator **Signature of the HOD**

ACKNOWLEDGEMENT

The success in this project would not have been possible but for the timely help and guidance rendered by many people. Our wish is to express our sincere thanks to all those who have assisted us in one way or the other for the completion of our project.

Our greatest appreciation goes to our Course Coordinator **Dr. D. Bheekya** and our guide **Dr. D. Bheekya**, Department of Computer Science and Engineering, whose tremendous support, encouragement and guidance for this project cannot be expressed in words.

We express our gratitude to **Dr. P Venkateshwara Rao (CSE)**, Head of the ***Department of Computer Science Engineering***, for providing us with adequate facilities, ways and means by which we were able to complete this project-based lab.

We thank all the members of the teaching and non-teaching staff, and also those who have assisted us directly or indirectly in the successful completion of this project. Finally, we sincerely thank our parents, friends and classmates for their kind help and cooperation during our work.

ILI HARSHIKA	**2520030607**
GURRAPU NIKSHITHA	**2520030512**
ANGELA JOSE	**2520030365**

Abstract

Process management is one of the most important functions of an Operating System, because every program that runs on a computer does so inside a process. Creating, executing, synchronizing and terminating processes correctly is what allows many programs to share a single machine safely. Yet for students, these ideas often remain abstract, because the system calls behind them are hidden inside commands that are used every day. To address this, our project presents the Linux Process Creation, Execution and Termination System, developed as ForgeOS in the C programming language using Linux/POSIX system calls. The core aim was to build a small, working system that demonstrates the complete life cycle of a process in a clear and practical manner.

ForgeOS is organised into two cooperating modules. ShellForge is the user command interface: it displays the ForgeOS prompt, reads the command typed by the user and hands it over for execution. NanoKernel is the process-management module: it creates a child process from the parent using the fork() system call, replaces the child with the requested program using the exec() family (execvp), and makes the parent wait for the child using wait()/waitpid(). Beneath both modules, the Linux kernel supplies the actual operating-system services.

After the child terminates, the parent retrieves the termination status using the macros WIFEXITED and WEXITSTATUS and displays a clear report that includes the parent and child process identifiers (PID and PPID), whether the termination was normal, the exit status and the final result (SUCCESS or FAILED). The system was compiled with GCC using the -Wall and -Wextra flags and tested on Ubuntu Linux, and all planned test cases passed. In summary, the project gives a transparent, working demonstration of process creation, program execution, synchronization and termination, and connects Operating System theory with practical system-level programming.

Index

S. No.	Chapter	Topics / Subtopics	Page No.
	Acknowledgement		**4**
	Abstract		**5**
1	**Introduction**		**8–9**
		1.1 Background of the Project	8
		1.2 Problem Statement	8
		1.3 Scope of the Project	8
		1.4 Objectives of the Project	9
		1.5 Importance of the Application	9
		1.6 Target Users / Audience	9
2	**OS Concepts and Background Study**		**10–11**
		2.1 Operating System Concepts Related to the Project	10
		2.2 Existing Methods / Algorithms	11
		2.3 Proposed Approach	11
		2.4 Advantages of the Proposed Approach	11
3	**System Requirements**		**12**
		3.1 Hardware Requirements	12
		3.2 Software Requirements	12
		3.3 Operating System Environment – Ubuntu / WSL	12
		3.4 Development Tools and Compilers	12
4	**Technology and Tools Used**		**13–14**
		4.1 C / C++ Programming	13
		4.2 GCC Compiler	13
		4.3 Linux System Calls	13
		4.4 POSIX Threads / Synchronization	13
		4.5 Linux Commands and Utilities	13
		4.6 Other Tools / Libraries Used	14
5	**System Architecture and Design**		**15–17**
		5.1 Overall System Architecture	15
		5.2 System Workflow	15
		5.3 Flowchart / Data Flow	16
		5.4 Module Description	17

S. No.	Chapter	Topics / Subtopics	Page No.
		5.5 Input and Output Design	17
6	**Implementation**		**18–22**
		6.1 Module-wise Implementation	18
		6.2 Algorithm / Pseudocode	18
		6.3 Core C / C++ Implementation	19
		6.4 Linux System Calls / APIs Used	21
		6.5 Process / Thread Implementation	21
		6.6 IPC / Synchronization Implementation	22
		6.7 Memory Management Implementation	22
		6.8 Optional User Interface Implementation	22
7	**Execution and Demonstration**		**23–26**
		7.1 Compilation in Ubuntu	23
		7.2 Execution Commands	23
		7.3 Sample Inputs and Outputs	23
		7.4 Screenshots of Execution	24
		7.5 Demonstration of OS Concepts	26
8	**Conclusion**		**27**
		8.1 Summary of the Project	27
		8.2 Objectives Achieved	27
		8.3 Results Achieved	27
		8.4 Skills and Learning Outcomes	27
9	**References**		**28**
		9.1 Books	28
		9.2 Research Papers	28
		9.3 Linux / GNU Documentation	28
		9.4 Online Resources	28

CHAPTER - 1 INTRODUCTION

1.1 Background of the Project

An Operating System (OS) is the layer of software that manages the hardware of a computer and provides services to the programs that run on it. One of its most fundamental services is process management. A process is a program in execution: it has its own address space, its own process identifier (PID), a set of open files and a current state. Whenever a user types a command in a terminal, opens an application or starts a background service, the OS creates a process to carry it out.

On Linux and other UNIX-like systems, processes are organised as a tree. A new process is created by an existing process (the parent) using the fork() system call, which produces a nearly identical copy called the child. The child can then load a different program using one of the exec() functions, while the parent can wait for the child to finish using wait() or waitpid(). When the child ends, it returns an exit status that the parent can read. This fork–exec–wait pattern is the foundation of every Linux shell, and understanding it is essential for any student of Operating Systems.

This project, named ForgeOS, implements that pattern in a small and readable form. It shows, step by step and with printed output, how a command given by the user becomes a child process, runs to completion and reports its termination status back to the parent.

1.2 Problem Statement

Process creation, execution, synchronization and termination are usually explained with diagrams and short code fragments, but students rarely see the whole life cycle running as one connected system. The problem addressed by this project is to develop a Linux-based system that practically demonstrates the complete process life cycle. The system must create a child process from a parent process using fork(), execute a selected program in the child using the exec() family of system calls, and allow the parent to wait for the child using wait(). After the child completes and terminates, the system must retrieve and display its termination status.

1.3 Scope of the Project

The scope of the project covers the following:

- A command-line interface (ShellForge) that accepts a program name with its arguments from the user.
- A process-management module (NanoKernel) that implements fork(), execvp() and waitpid() and reports every stage of the process life cycle.
- Display of PID and PPID values for the parent and child processes.
- Retrieval and interpretation of the child’s termination status (normal termination, exit code, success or failure).
- Implementation and testing on Ubuntu Linux using the GCC compiler and standard POSIX system calls.

The project does not attempt to build a complete operating system or a full-featured shell. Features such as pipes, input/output redirection, job control and background execution are outside the present scope and are listed as possible future work.

1.4 Objectives of the Project

- To implement process creation in Linux using the fork() system call and understand the parent–child process relationship.
- To execute a selected program in the child process using the exec() family of system calls.
- To synchronize the parent and child processes using the wait() system call.
- To demonstrate process termination, and to retrieve and display the termination status of the child process.

1.5 Importance of the Application

The application is important for both academic and practical reasons. Academically, it turns the abstract ideas of process management into a program whose behaviour can be observed directly: the printed PID and PPID values show the parent–child relationship, the "Parent waiting for child" message shows synchronization, and the final status report shows termination. Practically, the fork–exec–wait structure used here is the same structure used by real shells such as bash, by service managers and by many system utilities, so the knowledge gained transfers directly to real-world systems programming.

1.6 Target Users / Audience

- Undergraduate students learning Operating Systems and Systems Programming.
- Faculty members who want a compact, demonstrable example of the process life cycle for classroom use.
- Beginners in Linux system programming who want to understand how a shell launches commands.

CHAPTER - 2 OS CONCEPTS AND BACKGROUND STUDY

2.1 Operating System Concepts Related to the Project

Process and Process Control Block (PCB)

A process is a program in execution. For each process the OS maintains a data structure called the Process Control Block (PCB), which stores the process identifier, process state, program counter, CPU registers, memory-management information, open-file information and accounting data. The PCB is what allows the OS to suspend a process and later resume it, and it is the structure that the fork() system call duplicates (with a new PID) when a child is created.

Process States

During its life a process moves between states: New, Ready, Running, Waiting (Blocked) and Terminated. In this project the parent process enters the Waiting state when it calls waitpid(), and returns to Ready and then Running once the child terminates. The child moves from New to Ready on creation, runs the selected program, and finally enters the Terminated state.

Process Creation – fork()

The fork() system call creates a new process by duplicating the calling process. It is called once but returns twice: it returns 0 in the child, the PID of the child in the parent, and −1 if the creation fails. This return value is how the program decides which path (parent or child) to follow.

Program Execution – the exec() family

The exec() family (execl, execlp, execv, execvp, execve and others) replaces the current process image with a new program. The PID stays the same, but the code, data and stack are replaced by those of the new program. If exec succeeds it never returns; if it returns, an error has occurred. This project uses execvp(), which searches the directories listed in the PATH environment variable and accepts the arguments as an array.

Synchronization – wait() / waitpid()

The wait() and waitpid() system calls suspend the calling process until a child terminates. This synchronizes the parent with the child, ensures that the parent does not continue before the child has finished, and allows the parent to collect the child's exit status. It also prevents zombie processes: a child that has terminated but whose status has not been collected remains in the process table as a zombie until the parent reaps it.

Process Termination and Exit Status

A process terminates either normally (by returning from main() or calling exit()) or abnormally (when killed by a signal). The status value returned to the parent encodes this information. The macros WIFEXITED(status) and WEXITSTATUS(status) test for normal termination and extract the exit code, while WIFSIGNALED(status) and WTERMSIG(status) report termination by a signal. By convention, an exit code of 0 means success and a non-zero code means failure.

Process Identifiers – PID and PPID

Every process has a unique PID. Every process also records the PID of its parent (PPID). Displaying both values for the child and the parent makes the process tree visible. In the sample output of Chapter

7, the parent (ForgeOS) has PID 768 and each child created for a command has a different PID (772, 773) but the same PPID, 768.

2.2 Existing Methods / Algorithms

Several approaches exist for creating and running a new program on a Linux system:

- **fork() + exec() + wait():** the classical UNIX model. A shell forks a child, the child execs the command, and the shell waits. It is flexible and is used by almost all shells.
- **system():** a C library function that internally runs a shell to execute a command string. It is easy to use but hides the individual steps and gives the programmer little control.
- **popen():** runs a command and connects its input or output to the program through a pipe; it is useful for capturing output, but again hides the process life cycle.
- **posix_spawn():** a single call that combines creation and execution; efficient, but it does not expose the parent–child split that students need to study.
- **vfork():** an older optimisation in which the child temporarily shares the parent's memory until it calls exec; it is rarely recommended because of its subtle restrictions.

2.3 Proposed Approach

The project adopts the classical fork–exec–wait model because it exposes every stage of the life cycle. The approach is split into two modules to mirror the layered structure of a real system. ShellForge, the interface layer, reads the command and passes it on; NanoKernel, the process layer, performs creation, execution, synchronization and termination handling and prints a trace of each step. The Linux kernel underneath performs the real work through system calls.

The sequence followed for every command is: (1) the user enters a command; (2) the parent calls fork(); (3) the child calls execvp() to run the program; (4) the parent calls waitpid() and blocks; (5) the child terminates; (6) the parent examines the status using WIFEXITED and WEXITSTATUS; and (7) the result is displayed and control returns to the prompt.

2.4 Advantages of the Proposed Approach

- Transparency – each stage of the life cycle is printed, so the behaviour can be observed and explained.
- Modularity – the interface (ShellForge) and the process logic (NanoKernel) are separate, making the code easy to read, test and extend.
- Control – using the raw system calls gives full control over error handling and status reporting, unlike system() or popen().
- Realism – the fork–exec–wait pattern is exactly what real shells use, so the knowledge transfers directly.
- Light-weight – the project needs only a C compiler and a Linux environment; no external libraries are required.

CHAPTER - 3 SYSTEM REQUIREMENTS

3.1 Hardware Requirements

The program is very light-weight, so any modern computer is sufficient. The recommended minimum configuration is listed below.

Component	Requirement
Processor	Any modern 64-bit processor (Intel Core i3 / AMD Ryzen 3 or better)
RAM	4 GB minimum (8 GB recommended when running Ubuntu under WSL)
Storage	At least 2 GB of free disk space for the Linux environment, compiler and project files
Display / Input	Standard monitor, keyboard and mouse

3.2 Software Requirements

Software	Purpose
Ubuntu Linux (native or on WSL)	Operating system on which the project is compiled and executed
GCC (GNU Compiler Collection)	Compiles the C source files into the forgeos executable
GNU nano / VS Code	Editors used for writing the source code
GDB and terminal	Debugging, running and testing the program
Git and GitHub	Version control and the shared project repository (SystemForge_OSSP)

3.3 Operating System Environment – Ubuntu / WSL

The project relies on POSIX system calls (fork, execvp, waitpid, getpid, getppid), which are available on Linux but not on native Windows. The system was therefore developed and executed in an Ubuntu environment, using a standard Ubuntu terminal. On a Windows computer, the Windows Subsystem for Linux (WSL) provides a genuine Linux kernel interface and lets the same code compile and run without any change. The project files are kept in the directory structure SystemForge_OSSP/ForgeOS, which contains the two modules: NanoKernel and Shellforge.

3.4 Development Tools and Compilers

The C source code is compiled with GCC. The warning options -Wall and -Wextra are used so that the compiler reports suspicious constructs such as unused variables or missing return values. The two modules are compiled and linked together in a single command, producing the executable named forgeos. GDB can be used to step through the fork and exec calls, and the terminal is used for running and testing the program.

CHAPTER - 4 TECHNOLOGY AND TOOLS USED

4.1 C / C++ Programming

C is the natural language for systems programming on Linux, because the Linux system-call interface and the POSIX standard are defined in terms of C. It gives direct access to system calls, process identifiers and status values with very little overhead. The project is written in C, and the header file process.h is used to separate the interface of the NanoKernel module from its implementation.

4.2 GCC Compiler

GCC translates the C source into a native executable. The command used is:

```
gcc -Wall -Wextra shell.c ../NanoKernel/process.c -o forgeos
```

Here shell.c and process.c are the two source files, -o names the output file and -Wall -Wextra turn on additional warnings. The relative path ../NanoKernel/process.c shows that the two modules live in separate directories and are linked at compile time.

4.3 Linux System Calls

System calls are the entry points through which a user program requests services from the kernel. The calls used in this project are summarised below.

System call	Header	Purpose in the project
fork()	<unistd.h>	Creates a new child process that is a copy of the parent
execvp()	<unistd.h>	Replaces the child's process image with the requested program
waitpid()	<sys/wait.h>	Suspends the parent until the child terminates and collects its status
getpid()	<unistd.h>	Returns the PID of the calling process
getppid()	<unistd.h>	Returns the PID of the parent of the calling process
exit()	<stdlib.h>	Terminates the child if exec fails, returning an error status
getcwd()	<unistd.h>	Obtains the current directory to show in the ForgeOS prompt

4.4 POSIX Threads / Synchronization

POSIX threads (pthreads) provide multiple flows of control inside one process, together with mutexes and semaphores for synchronizing them. This project does not use threads, because its subject is multiple processes rather than multiple threads. The synchronization used here is process-level: the parent is synchronized with the child through waitpid(), which blocks the parent until the child changes state. This is the simplest and most important form of synchronization in the fork–exec–wait model.

4.5 Linux Commands and Utilities

Command	Use in the project
mkdir -p, mkdir	Creating the project directory structure (ForgeOS, Shellforge, NanoKernel)

Command	Use in the project
cd, pwd	Moving between directories and confirming the current location
ls, ls -a	Listing files; also used as sample commands to test ForgeOS
nano	Writing process.h, process.c and shell.c
gcc	Compiling and linking the source files
./forgeos	Running the compiled program

4.6 Other Tools / Libraries Used

- **Standard C library:** stdio.h (printf, fgets, perror), stdlib.h (exit) and string.h (strcmp, strtok, strcspn).
- **POSIX headers:** unistd.h, sys/types.h and sys/wait.h for the process-related calls and status macros.
- **VS Code:** used for organising and editing the project source code.
- **Git and GitHub:** used to maintain the SystemForge_OSSP repository shared among the team members.

CHAPTER - 5 SYSTEM ARCHITECTURE AND DESIGN

5.1 Overall System Architecture

ForgeOS follows a layered architecture with four layers, as shown in Figure 5.1. The user interacts only with ShellForge. ShellForge passes each request to NanoKernel, and NanoKernel uses Linux system calls to create and manage the process. The results flow back up the same path so that the user sees them at the prompt.

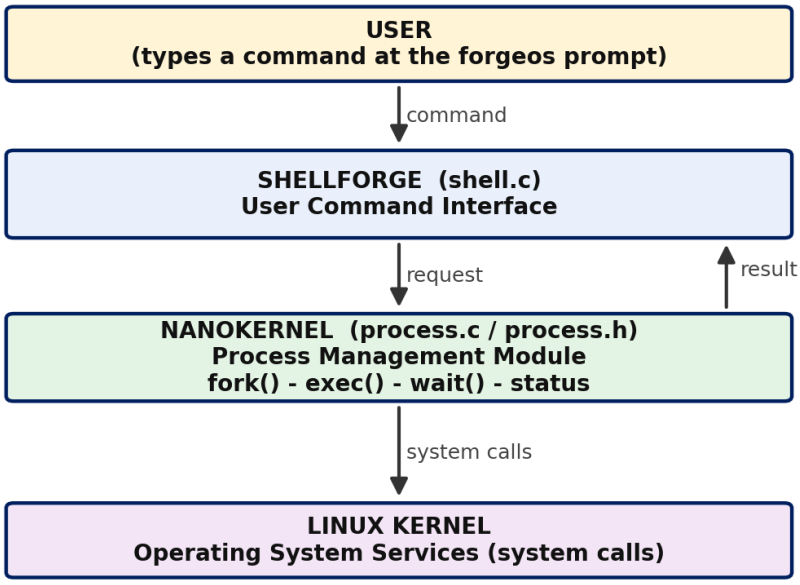


Figure 5.1: Overall architecture of ForgeOS

Layer	Responsibility
User	Types commands at the forgeos prompt and reads the results
ShellForge (shell.c)	Displays the banner and prompt, reads and parses the command, handles built-in words such as help and exit, and calls NanoKernel
NanoKernel (process.c)	Creates the child (fork), runs the program (execvp), synchronizes (waitpid), interprets the exit status and prints the result
Linux Kernel	Provides the real operating-system services: process table, scheduling, memory and program loading

5.2 System Workflow

The workflow for one command is as follows:

1. The user starts ./forgeos and the ForgeOS banner is displayed.
2. At the prompt forgeos:<current directory>$ the user types a command, for example ls -a.
3. ShellForge prints “Command received”, splits the command into arguments and calls the NanoKernel module.
4. NanoKernel prints “Request received” and calls fork(). Linux creates the child process.
5. The parent prints its own PID and the child’s PID, then calls waitpid() and blocks.

6. The child prints “Child process created”, shows its PID and PPID, and calls execvp(). The child image is replaced by the requested program, which runs and prints its output.

7. When the program finishes, the child terminates and the parent is unblocked.

8. The parent checks the status with WIFEXITED and WEXITSTATUS and prints “Normal termination”, “Exit status” and “Result”.

9. Control returns to ShellForge, which shows the next prompt. Typing exit shuts ForgeOS down.

5.3 Flowchart / Data Flow

Figure 5.2 shows the overall flow of the system, and Figure 5.3 shows how the parent and child processes run in time. The key point is that fork() splits the flow into two paths: the parent blocks in waitpid(), while the child executes the program; the two paths join again when the child terminates and the parent retrieves its status.

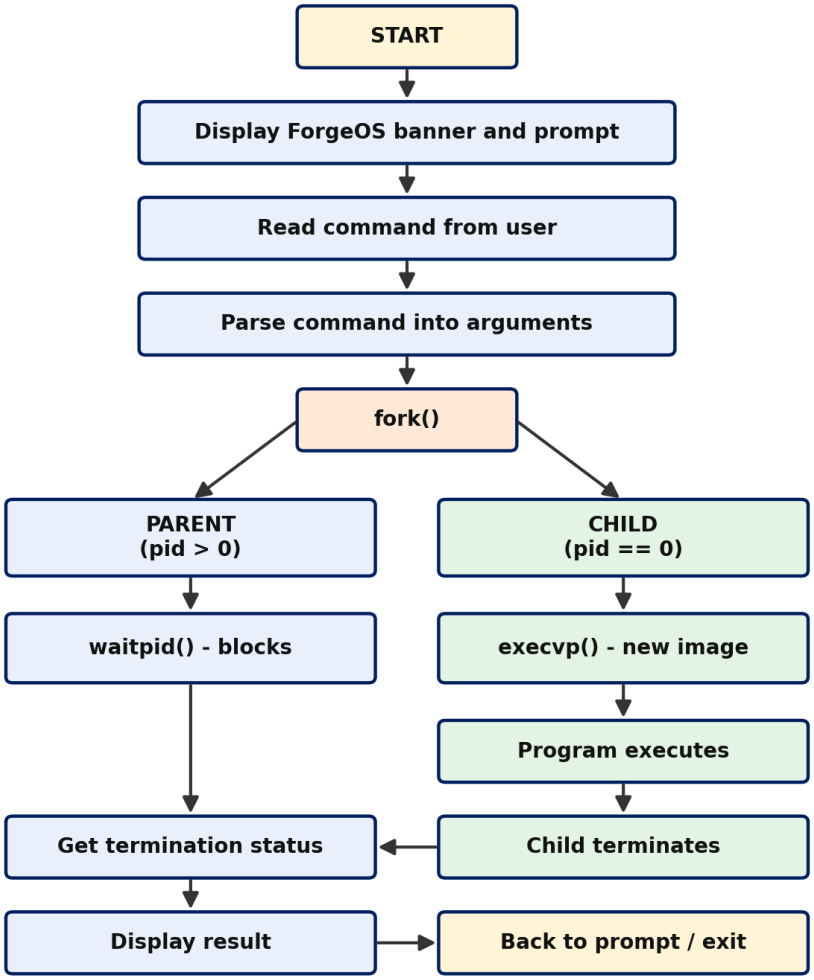


Figure 5.2: Flowchart of the process creation, execution and termination system

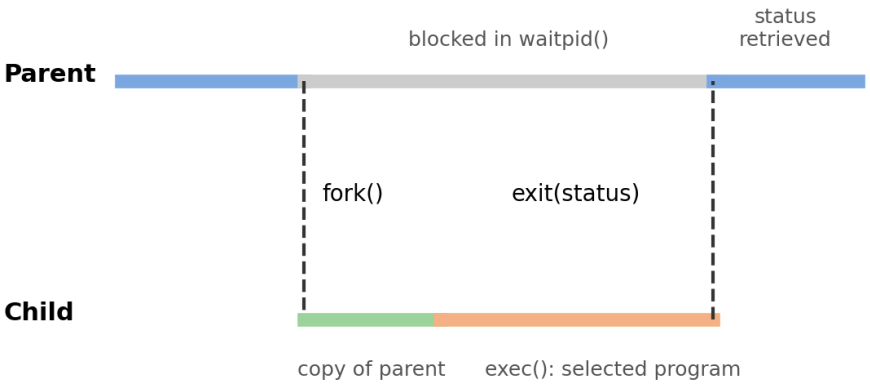


Figure 5.3: Parent and child timeline

5.4 Module Description

Module 1 – ShellForge (User Command Interface)

ShellForge is implemented in shell.c. It prints the ForgeOS banner, then repeatedly shows the prompt, reads a line with fgets(), removes the trailing newline, splits the line into an argument array with strtok(), and calls NanoKernel. It also recognises the words help (to list command categories) and exit (to shut ForgeOS down).

Module 2 – NanoKernel (Process Management Module)

NanoKernel is implemented in process.c with its interface in process.h. Its single entry function, execute_command(), carries out the whole life cycle: creation with fork(), execution with execvp(), synchronization with waitpid(), and termination handling with WIFEXITED and WEXITSTATUS. It prints a message at each stage.

Module 3 – Linux Kernel Services

This is not code written by the team, but it is an essential part of the design. The kernel implements fork, exec and wait, maintains the process table (the PCBs), schedules the processes and loads the new program image.

5.5 Input and Output Design

Input

- A command line typed at the prompt: a program name followed by optional arguments, for example pwd, ls or ls -a.
- The words help and exit, which are handled by ShellForge itself.

Output

- ShellForge message: “[ShellForge] Command received: <command>”.
- NanoKernel trace messages: request received, creating child process, parent and child PIDs, parent waiting, child created, executing using execvp(), and the command name.
- The output of the executed program itself.
- Termination report: child process terminated, normal termination (YES/NO), exit status, and result (SUCCESS/FAILED).

CHAPTER - 6 IMPLEMENTATION

6.1 Module-wise Implementation

The project directory is organised as follows:

```
SystemForge_OSSP/
└── ForgeOS/
    ├── NanoKernel/
    │   ├── process.h      (interface of the process module)
    │   └── process.c      (fork / exec / wait / status)
    └── Shellforge/
        └── shell.c        (command interface and main loop)
```

The work was divided among the team members according to the stages of the process life cycle, as shown below.

Roll Number	Student Name	Individual Responsibility
2520030607	Ili Harshika	Process creation using fork() and handling of the parent–child process relationship.
2520030512	Gurrapu Nikshitha	Program execution using exec() and process synchronization using wait().
2520030365	Angela Jose	Process termination, exit-status handling, and testing of the system.

6.2 Algorithm / Pseudocode

Algorithm for ShellForge

```
1.  Display the ForgeOS banner
2.  Repeat:
      a. Display the prompt "forgeos:<cwd>$"
      b. Read a line of input; if end-of-input, stop
      c. If the line is empty, go to step 2
      d. If the line is "exit", display shutdown message and stop
      e. If the line is "help", display the help text and go to step 2
      f. Print "[ShellForge] Command received: <line>"
      g. Split the line into arguments args[0..n-1]; set args[n] = NULL
      h. Call execute_command(args)
```

Algorithm for NanoKernel – execute_command(args)

```
1.  Print "Request received" and "Creating child process..."
2.  pid = fork()
3.  If pid < 0:
        report the error and return
4.  If pid == 0:                        /* CHILD */
        print "Child process created", own PID and PPID
        print "Executing using execvp()..." and the command
        execvp(args[0], args)
        if execvp returns: report error and exit(127)
5.  Else:                               /* PARENT */
        print parent PID, child PID, "Parent waiting for child..."
        waitpid(pid, &status, 0)
6.  Print "Child process terminated"
7.  If WIFEXITED(status):
        print "Normal termination : YES"
        code = WEXITSTATUS(status)
        print "Exit status : code"
        print "Result : SUCCESS" if code == 0 else "FAILED"
    Else if WIFSIGNALED(status):
        print "Normal termination : NO" and the signal number
8.  Return to ShellForge
```

6.3 Core C / C++ Implementation

The two core files of the project are given below. process.h declares the interface of the NanoKernel module:

```
/* process.h – NanoKernel interface */
#ifndef PROCESS_H
#define PROCESS_H

/* Creates a child, runs args[0] with args[], waits for it, and
   displays the termination status. Returns the raw wait status,
   or -1 if the process could not be created. */
int execute_command(char *args[]);

#endif
```

process.c contains the process life-cycle logic:

```
/* process.c – NanoKernel: process management module */
#include <stdio.h>
#include <stdlib.h>
#include <unistd.h>
#include <sys/types.h>
#include <sys/wait.h>
#include "process.h"

int execute_command(char *args[])
{
    pid_t pid;
    int status;

    printf("\n[NanoKernel] Request received.\n");
    printf("[NanoKernel] Creating child process...\n");

    pid = fork();                               /* create child */

    if (pid < 0) {                              /* fork failed */
        perror("[NanoKernel] fork failed");
        return -1;
    }
    else if (pid == 0) {                        /* ---- CHILD ---- */
        printf("[NanoKernel] Child process created.\n");
        printf("[NanoKernel] Child PID  : %d\n", getpid());
        printf("[NanoKernel] Parent PID : %d\n", getppid());
        printf("[NanoKernel] Executing using execvp()...\n");
        printf("[NanoKernel] Command     : %s\n\n", args[0]);
        fflush(stdout);

        execvp(args[0], args);                  /* replace image */
        perror("[NanoKernel] execvp failed");  /* only if exec fails */
        exit(127);
    }
    else {                                      /* ---- PARENT ---- */
        printf("[NanoKernel] Parent PID : %d\n", getpid());
        printf("[NanoKernel] Child PID  : %d\n", pid);
        printf("[NanoKernel] Parent waiting for child...\n");
        fflush(stdout);

        waitpid(pid, &status, 0);               /* synchronize */

        printf("\n[NanoKernel] Child process terminated.\n");
        if (WIFEXITED(status)) {
            int code = WEXITSTATUS(status);
            printf("[NanoKernel] Normal termination : YES\n");
            printf("[NanoKernel] Exit status         : %d\n", code);
            printf("[NanoKernel] Result              : %s\n",
                   code == 0 ? "SUCCESS" : "FAILED");
```

```
        } else if (WIFSIGNALED(status)) {
            printf("[NanoKernel] Normal termination : NO\n");
            printf("[NanoKernel] Killed by signal    : %d\n", WTERMSIG(status));
            printf("[NanoKernel] Result              : FAILED\n");
        }
    }
    return status;
}
```

shell.c contains the ShellForge command interface:

```
/* shell.c - ShellForge: user command interface */
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <unistd.h>
#include "../NanoKernel/process.h"

#define MAX_INPUT 256
#define MAX_ARGS   32

static void print_banner(void)
{
    printf("\n=====================================================\n");
    printf("                      FORGE OS\n");
    printf("              PROCESS MANAGEMENT SYSTEM\n");
    printf("=====================================================\n");
    printf(" ShellForge    : User Command Interface\n");
    printf(" NanoKernel    : Process Management Module\n");
    printf(" Linux Kernel  : Operating System Services\n");
    printf("=====================================================\n\n");
    printf("Type 'help' to see the available command categories.\n");
    printf("Type 'exit' to shut down ForgeOS.\n\n");
}

int main(void)
{
    char input[MAX_INPUT];
    char *args[MAX_ARGS];
    char cwd[512];

    print_banner();

    while (1) {
        getcwd(cwd, sizeof(cwd));
        printf("forgeos:%s$ ", cwd);

        if (fgets(input, sizeof(input), stdin) == NULL)
            break;
        input[strcspn(input, "\n")] = '\0';

        if (strlen(input) == 0)
            continue;
        if (strcmp(input, "exit") == 0) {
            printf("Shutting down ForgeOS...\n");
            break;
        }

        printf("\n[ShellForge] Command received: %s\n", input);

        int n = 0;
        char *tok = strtok(input, " ");
        while (tok != NULL && n < MAX_ARGS - 1) {
            args[n++] = tok;
            tok = strtok(NULL, " ");
        }
        args[n] = NULL;
```

```
        execute_command(args);
    }
    return 0;
}
```

6.4 Linux System Calls / APIs Used

System call / API	How it is used	Concept demonstrated
fork()	Called by NanoKernel to create the child; its return value (<0, 0, >0) selects the failure, child or parent branch	Process creation, parent–child relationship
execvp(file, argv)	Called in the child with the command name and argument array; searches PATH	Program execution, replacing the process image
waitpid(pid, &status, 0)	Called in the parent to block until the child terminates and to collect the status	Synchronization, zombie prevention
WIFEXITED / WEXITSTATUS	Tests for normal termination and extracts the exit code	Termination status
WIFSIGNALED / WTERMSIG	Detects and reports termination by a signal	Abnormal termination
getpid() / getppid()	Prints the PID and PPID of the processes	Process identifiers
exit(127)	Ends the child if execvp fails	Error handling

6.5 Process / Thread Implementation

Every command typed by the user is executed in a new child process created by fork(). The ForgeOS shell process is the parent and remains alive between commands; each child lives only for the duration of one command. Immediately after fork(), the parent and child are two separate processes with separate PIDs; they run the same code but follow different branches because of the value returned by fork(). The child then calls execvp(), after which it is no longer a copy of ForgeOS but the requested program.

In the sample run, the ForgeOS parent had PID 768, and the children created for pwd and ls -a had PIDs 772 and 773. All children report the same PPID (768), which confirms that they were all created by the same parent. No threads are created; the project is single-threaded and the concurrency demonstrated is between processes.

6.6 IPC / Synchronization Implementation

Synchronization between the parent and child is achieved with waitpid(). Without it, the parent would print the next prompt while the child was still running and the outputs would be interleaved. With waitpid(), the parent is blocked until the child terminates, and the child's exit status is passed back to the parent through the kernel as the status value.

The only inter-process communication in this project is this exit-status return value. Other IPC mechanisms such as pipes, shared memory, message queues and sockets are not needed, and are noted as possible future extensions (for example, implementing the shell pipe operator).

6.7 Memory Management Implementation

The project does not allocate memory dynamically; it uses fixed-size arrays for the input line and the argument list. Memory management is nevertheless involved in the process life cycle. On fork(), Linux gives the child a copy of the parent's address space, implemented efficiently with copy-on-write: the pages are shared until either process modifies them. On execvp(), the child's old address space is discarded and a new one is built for the loaded program. When the child terminates, the kernel frees its memory, and the parent's waitpid() releases the remaining process-table entry.

6.8 Optional User Interface Implementation

ShellForge provides a text-based user interface. It displays a banner describing the three layers of ForgeOS, shows a prompt that includes the current working directory (for example forgeos:/home/nikshitha/SystemForge_OSSP/ForgeOS/Shellforge$), prints a clear label before each message so that the origin of the output ([ShellForge] or [NanoKernel]) is always known, and accepts the commands help and exit. No graphical interface was developed, because the goal of the project is to expose the process life cycle rather than to provide a graphical environment.

CHAPTER - 7 EXECUTION AND DEMONSTRATION

7.1 Compilation in Ubuntu

The project directories were created and the source files written in the Ubuntu terminal. The commands used were:

```
cd SystemForge_OSSP
cd ForgeOS
mkdir -p Shellforge
mkdir NanoKernel
cd NanoKernel
nano process.h
nano process.c
cd ..
cd Shellforge
nano shell.c
gcc -Wall -Wextra shell.c ../NanoKernel/process.c -o forgeos
```

The compilation completed without errors, and the executable forgeos was created in the Shellforge directory.

7.2 Execution Commands

```
cd ~/SystemForge_OSSP/ForgeOS/Shellforge
./forgeos
```

After the ForgeOS banner appears, commands such as ls, pwd and ls -a are typed at the forgeos prompt. The command exit shuts ForgeOS down.

7.3 Sample Inputs and Outputs

Sample Input 1: pwd

```
forgeos:/home/nikshitha/SystemForge_OSSP/ForgeOS/Shellforge$ pwd

[ShellForge] Command received: pwd

[NanoKernel] Request received.
[NanoKernel] Creating child process...
[NanoKernel] Parent PID : 768
[NanoKernel] Child PID  : 772
[NanoKernel] Parent waiting for child...
[NanoKernel] Child process created.
[NanoKernel] Child PID  : 772
[NanoKernel] Parent PID : 768
[NanoKernel] Executing using execvp()...
[NanoKernel] Command     : pwd

/home/nikshitha/SystemForge_OSSP/ForgeOS/Shellforge

[NanoKernel] Child process terminated.
[NanoKernel] Normal termination : YES
[NanoKernel] Exit status        : 0
[NanoKernel] Result             : SUCCESS
```

Sample Input 2: ls -a

```
forgeos:/home/nikshitha/SystemForge_OSSP/ForgeOS/Shellforge$ ls -a

[ShellForge] Command received: ls -a

[NanoKernel] Request received.
[NanoKernel] Creating child process...
[NanoKernel] Parent PID : 768
[NanoKernel] Child PID  : 773
```

```
[NanoKernel] Parent waiting for child...
[NanoKernel] Child process created.
[NanoKernel] Child PID  : 773
[NanoKernel] Parent PID : 768
[NanoKernel] Executing using execvp()...
[NanoKernel] Command    : ls

.  ..  forgeos  shell.c

[NanoKernel] Child process terminated.
[NanoKernel] Normal termination : YES
[NanoKernel] Exit status        : 0
[NanoKernel] Result             : SUCCESS
```

In both runs the parent PID is the same (768), the child PIDs are different (772 and 773), the program output appears between the “Executing” message and the “terminated” message, and the exit status 0 gives the result SUCCESS.

Test cases

ID	Function tested	Expected result	Result
TC01	Process creation	Child process is created successfully.	PASS
TC02	Program execution	Selected program executes through the child process.	PASS
TC03	Synchronization	Parent waits for the child to terminate.	PASS
TC04	Normal termination	Child terminates and status is retrieved.	PASS
TC05	Exit status	Termination status is retrieved and displayed.	PASS

7.4 Screenshots of Execution

Figure 7.1 shows the creation of the project directories, the compilation with GCC and the start of ForgeOS, with the ForgeOS banner and the first command (ls) received by ShellForge.

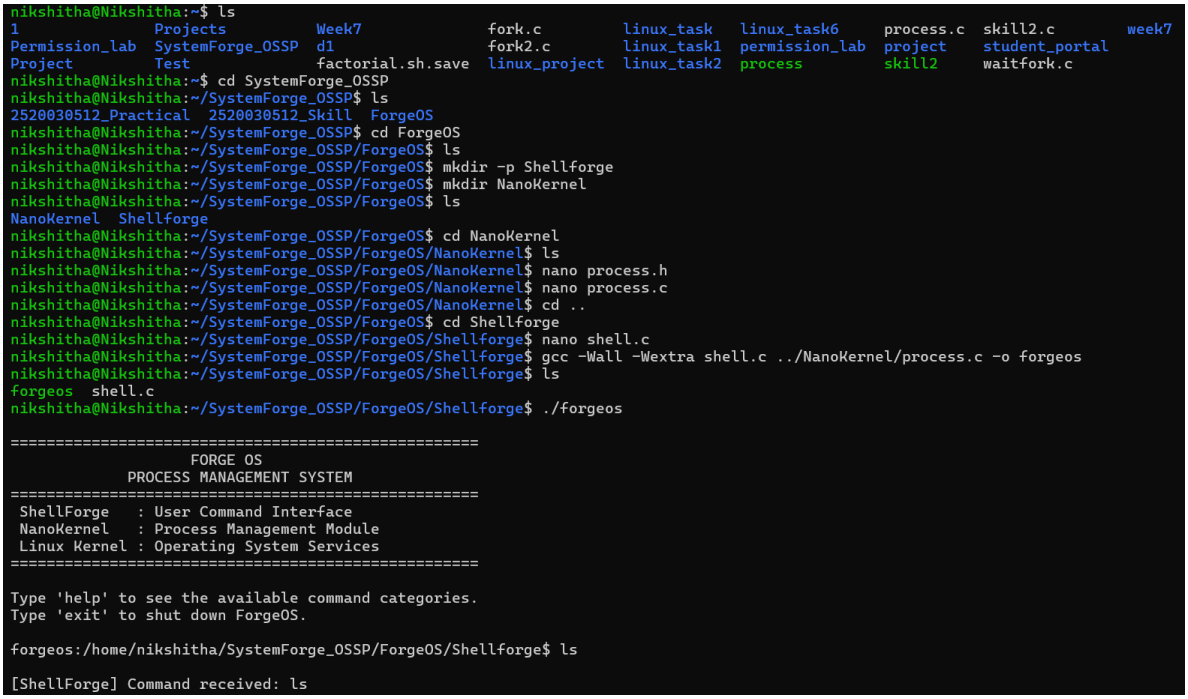


Figure 7.1: Directory setup, compilation and start of ForgeOS

Figure 7.2 shows the NanoKernel trace for the commands pwd and ls -a, including the parent and child PIDs, the execution of the program and the termination report.

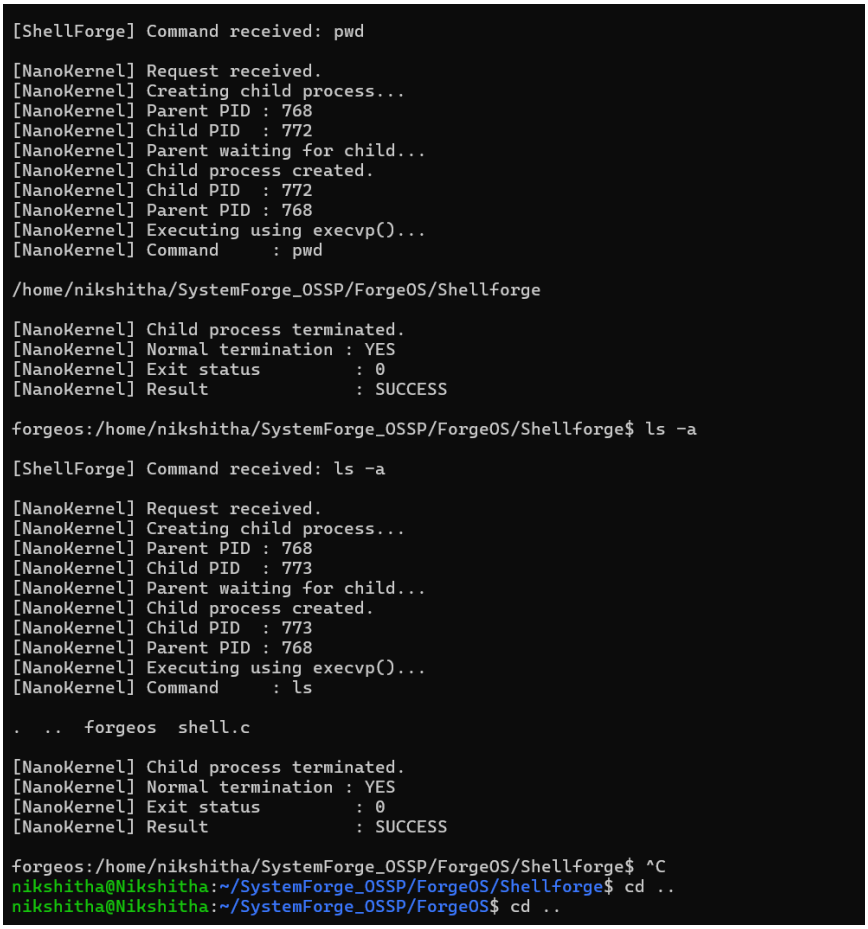


Figure 7.2: NanoKernel output for pwd and ls -a

7.5 Demonstration of OS Concepts

OS concept	Where it is visible in the output
Process creation (fork)	"Creating child process…" followed by two different PIDs (parent 768, child 772/773).
Parent–child relationship	The child reports Parent PID 768, which equals the parent's own PID.
Program execution (exec)	"Executing using execvp()…" and the output of pwd / ls appears from the child.
Synchronization (wait)	"Parent waiting for child…" is printed, and the next prompt appears only after the child has terminated.
Process termination	"Child process terminated." is printed after the program's output.
Exit status	"Normal termination : YES", "Exit status : 0" and "Result : SUCCESS".

CHAPTER - 8 CONCLUSION

8.1 Summary of the Project

The Linux Process Creation, Execution and Termination System (ForgeOS) was designed, implemented and tested successfully. The system uses a layered design in which ShellForge accepts the user's command, NanoKernel creates and manages the process, and the Linux kernel provides the underlying services. For every command, a child is created with fork(), the program is executed with execvp(), the parent is synchronized with waitpid(), and the termination status is retrieved and displayed.

8.2 Objectives Achieved

Objective	Concept used	Status
Implement process creation using fork() and understand the parent–child relationship	fork(), PID, PPID	Achieved
Execute a selected program in the child using the exec() family	execvp()	Achieved
Synchronize parent and child using wait()	waitpid()	Achieved
Demonstrate process termination and display the child's termination status	WIFEXITED, WEXITSTATUS	Achieved

8.3 Results Achieved

- A working two-module system (ShellForge and NanoKernel) was compiled with GCC without errors and executed on Ubuntu.
- Commands such as pwd and ls -a were launched through child processes, and their output was displayed correctly.
- The parent and child PIDs were shown, proving that each command runs in a separate process created by the same parent.
- The parent waited for each child and reported normal termination, exit status 0 and the result SUCCESS.
- All five test cases (TC01–TC05) passed.

8.4 Skills and Learning Outcomes

- Practical understanding of the fork–exec–wait model and the life cycle of a Linux process.
- Experience using POSIX system calls and interpreting exit status with WIFEXITED and WEXITSTATUS.
- Modular C programming with header files and separate compilation of multiple source files.
- Use of the Linux terminal, nano, GCC and Git/GitHub for development and teamwork.
- Skills in testing, documenting and presenting a systems project as a team.

Future scope. The system can be extended with background execution (&), input/output redirection, pipes between commands, built-in commands such as cd, signal handling (for example Ctrl+C), and the use of threads or other IPC mechanisms.

CHAPTER - 9 REFERENCES

9.1 Books

[1] A. Silberschatz, P. B. Galvin and G. Gagne, Operating System Concepts, Wiley.

[2] W. R. Stevens and S. A. Rago, Advanced Programming in the UNIX Environment, Addison-Wesley.

[3] M. Kerrisk, The Linux Programming Interface, No Starch Press, 2010.

[4] A. S. Tanenbaum and H. Bos, Modern Operating Systems, Pearson.

[5] B. W. Kernighan and D. M. Ritchie, The C Programming Language, Prentice Hall.

9.2 Research Papers

[1] A. Baumann, J. Appavoo, O. Krieger and T. Roscoe, "A fork() in the road," Proceedings of the Workshop on Hot Topics in Operating Systems (HotOS), 2019.

[2] D. M. Ritchie and K. Thompson, "The UNIX Time-Sharing System," Communications of the ACM, vol. 17, no. 7, 1974.

9.3 Linux / GNU Documentation

[1] Linux manual pages: fork(2), execve(2), exec(3), wait(2), getpid(2).

[2] The GNU C Library Reference Manual – Processes.

[3] GCC, the GNU Compiler Collection – Online Documentation.

9.4 Online Resources

[1] Linux man-pages project: https://man7.org/linux/man-pages/

[2] GNU Project documentation: https://www.gnu.org/software/libc/manual/

[3] Project repositories: https://github.com/iliharshika/SystemForge_OSSP, https://github.com/gurrapunikshitha/SystemForge_OSSP and https://github.com/angelapvjose92/SystemForge_OSSP